

GRASP para a otimização do Índice de Qualidade Caminhável (IQC): método clássico, implementação e análise dos resultados

1 GRASP

Esta seção descreve o GRASP na sua forma clássica, seguindo a apresentação de Resende e Ribeiro [1]. O objetivo aqui é registrar o método como ele foi originalmente proposto, sem ainda tratar do problema específico desta tese. As decisões de adaptação ficam para a Seção 2, e a análise dos resultados obtidos com os dados do projeto, para a Seção 3.

1.1 Visão geral

GRASP é a sigla de *greedy randomized adaptive search procedures*. Trata-se de uma metaheurística multi-start (de partidas múltiplas) voltada a problemas de otimização combinatória. Considera-se o problema de minimizar (ou maximizar) uma função objetivo $f(S)$ sobre um conjunto de soluções viáveis $S \in X$, definido a partir de um conjunto-base (ground set) $E = \{e_1, \dots, e_n\}$, de uma família de soluções viáveis $X \subseteq 2^E$ e da função $f : 2^E \rightarrow \mathbb{R}$. Procura-se uma solução S^* tal que $f(S^*) \leq f(S)$ para todo $S \in X$, no caso de minimização.

Cada iteração do GRASP tem duas fases. A primeira é a construção, que produz uma solução viável a partir do zero. A segunda é a busca local, que investiga a vizinhança dessa solução até encontrar um ótimo local. As duas fases são repetidas muitas vezes, e a melhor solução encontrada ao longo de todas as iterações é retornada. Quando a construção gera uma solução inviável, aplica-se um procedimento de reparo; se a viabilidade não for alcançada, a solução é descartada e outra é gerada.

Uma característica frequentemente citada do método é a facilidade de implementação. São poucos os parâmetros a ajustar, o que permite concentrar o esforço de desenvolvimento em estruturas de dados que tornem cada iteração rápida. Em sua forma básica, o GRASP depende essencialmente de dois parâmetros: o número de iterações e o parâmetro que limita o tamanho da lista restrita de candidatos usada na construção.

1.2 Construção gulosa e aleatorização

A fase de construção parte de um algoritmo guloso. Em um algoritmo guloso puro, a solução é montada elemento a elemento: a cada passo, avalia-se o custo incremental de incorporar cada candidato do conjunto-base à solução parcial, e escolhe-se o elemento de menor custo incremental (no caso de minimização), atualizando-se em seguida o conjunto de candidatos e os custos. O resultado não é necessariamente ótimo, mas serve como ponto de partida para busca local ou para metaheurísticas.

O GRASP substitui a escolha estritamente gulosa por uma escolha gulosa e aleatorizada. A cada passo, monta-se uma lista restrita de candidatos, chamada RCL (*restricted candidate list*), formada pelos melhores elementos segundo a função gulosa. O próximo elemento a ser incorporado é sorteado dentro dessa lista, em vez de ser sempre o de melhor custo incremental. Como o sorteio muda a cada execução, o mesmo procedimento produz soluções diferentes em execuções diferentes, o que é a base do caráter multi-start do método.

Há duas formas de limitar a RCL. Na versão baseada em cardinalidade, a lista guarda os p elementos de melhor custo incremental, sendo p um parâmetro. Na versão baseada em qualidade (ou valor), usa-se um parâmetro de limiar $\alpha \in [0, 1]$. Denotando por $c(e)$ o custo incremental de incorporar o elemento e , e por $c^{\min}$ e $c^{\max}$ o menor e o maior custos incrementais entre os candidatos, a RCL para minimização é

$$\text{RCL} = \{ e \in C : c(e) \leq c^{\min} + \alpha (c^{\max} - c^{\min}) \}. \quad (1)$$

Nessa formulação, $\alpha = 0$ corresponde a um algoritmo guloso puro, pois só o melhor elemento entra na lista, enquanto $\alpha = 1$ corresponde a uma construção puramente aleatória, já que todos os candidatos passam a ser aceitos. O Algoritmo 1 reproduz a construção com o limiar α .

Linha a linha, o Algoritmo 1 funciona assim. A linha 2 inicia a solução vazia. A linha 3 coloca todos os elementos do conjunto-base como candidatos. A linha 4 calcula o custo incremental de cada candidato. O laço das linhas 5 a 11 repete-se enquanto houver candidatos. As linhas 6 e 7 obtêm o menor e o maior custo incremental do momento, que definem a faixa usada no limiar. A linha 8 monta a RCL com todos os candidatos cujo custo esteja dentro da fração α da faixa. A linha 9 sorteia um elemento da RCL, e a linha 10 o incorpora à solução. A linha 11 atualiza a lista de candidatos e recalcula os custos, porque incorporar um elemento pode mudar o custo incremental dos demais. Ao final, a linha 13 devolve a solução construída.

O GRASP pode ser entendido como uma técnica de amostragem repetitiva. Cada iteração produz uma amostra de solução vinda de uma distribuição desconhecida, cuja média e variância dependem de quão restritiva é a RCL. Valores pequenos de α levam a soluções com média próxima da solução

Algorithm 1 Construção gulosa e aleatorizada (minimização)

```
1: procedure CONSTRUCAOGULOSAALAEATORIZADA( $\alpha$ , semente)
2:    $S \leftarrow \emptyset$ 
3:    $C \leftarrow E$   $\triangleright$  conjunto de candidatos
4:   avalie o custo incremental  $c(e)$  para todo  $e \in C$ 
5:   while  $C \neq \emptyset$  do
6:      $c^{\min} \leftarrow \min\{c(e) : e \in C\}$ 
7:      $c^{\max} \leftarrow \max\{c(e) : e \in C\}$ 
8:      $RCL \leftarrow \{e \in C : c(e) \leq c^{\min} + \alpha(c^{\max} - c^{\min})\}$ 
9:     escolha  $s$  ao acaso na RCL
10:     $S \leftarrow S \cup \{s\}$ 
11:    atualize  $C$  e reavalie  $c(e)$  para todo  $e \in C$ 
12:  end while
13:  return  $S$ 
14: end procedure
```

gulosa e pouca dispersão; valores maiores aumentam a dispersão. Observou-se que costuma dar bons resultados a combinação de média relativamente baixa com variância relativamente alta, situação em que se enquadra, por exemplo, $\alpha = 0,2$.

1.3 Vizinhanças e busca local

Uma vizinhança de uma solução S é um conjunto $N(S) \subseteq X$. Cada solução $S' \in N(S)$ é alcançada a partir de S por uma operação chamada movimento, e em geral duas soluções vizinhas diferem por poucos elementos. Uma solução S^* é um ótimo local em relação a N quando $f(S^*) \leq f(S)$ para todo $S \in N(S^*)$.

A busca local tenta melhorar a solução de forma iterativa, trocando a solução corrente por uma solução melhor da vizinhança, até que nenhuma solução melhor seja encontrada. A eficácia do procedimento depende da estrutura de vizinhança, da técnica de varredura, da rapidez com que a função objetivo é avaliada e da solução de partida. A varredura pode ser feita de duas maneiras. Na estratégia de melhor melhora (*best-improving*), todos os vizinhos são examinados e a solução corrente é substituída pelo melhor deles. Na estratégia de primeira melhora (*first-improving*), a solução se move para o primeiro vizinho de custo menor que o corrente. As duas costumam chegar a soluções de qualidade semelhante, mas a primeira melhora tende a gastar menos tempo, e a melhor melhora é mais propensa a convergência prematura para um ótimo local não global.

Quando a vizinhança é muito grande ou a avaliação de seus elementos é cara, empregam-se estratégias de restrição de vizinhança e listas de candidatos, cujo propósito é isolar regiões promissoras e examinar apenas parte dos

Algorithm 2 Template do GRASP (minimização)

```
1: procedure GRASP(MaxIter, semente)
2:    $f^* \leftarrow +\infty$ 
3:   for  $k = 1, \dots, \text{MaxIter}$  do
4:      $S \leftarrow \text{CONSTRUCAOGULOSA ALEATORIZADA}(\alpha, \text{semente})$ 
5:     if  $S$  não é viável then
6:        $S \leftarrow \text{REPARO}(S)$ 
7:     end if
8:      $S \leftarrow \text{BUSCALOCAL}(S)$ 
9:     if  $f(S) < f^*$  then
10:       $S^* \leftarrow S$ ;  $f^* \leftarrow f(S)$ 
11:    end if
12:  end for
13:  return  $S^*$ 
14: end procedure
```

movimentos. A eficiência dessas estratégias pode ser reforçada por estimativas de valor de movimento calculadas de forma rápida, usadas para descartar movimentos com estimativas ruins antes de uma avaliação completa. Esse ponto será retomado na Seção 2, porque a construção usada neste trabalho se apoia exatamente em uma estimativa desse tipo.

1.4 O laço multi-start e o template

O template básico do GRASP aparece no Algoritmo 2. Ele executa um número fixo de iterações. Em cada iteração, gera uma solução pela construção gulosa e aleatorizada, repara-a se necessário, aplica busca local e atualiza a melhor solução conhecida. O parâmetro *semente* inicializa o gerador de números pseudoaleatórios, o que permite reproduzir uma execução.

No Algoritmo 2, a linha 2 inicia o melhor valor conhecido como infinito, já que é minimização. O laço das linhas 3 a 10 executa **MaxIter** iterações independentes. A linha 4 constrói uma solução. As linhas 5 a 7 reparam a solução se ela for inviável. A linha 8 aplica a busca local. As linhas 9 e 10 guardam a solução como melhor conhecida quando seu valor supera o melhor até então. A linha 12 devolve a melhor solução de todas as iterações. Um ponto importante é que, na forma original, o GRASP não guarda informação entre iterações: cada iteração é independente das demais. Essa independência é o que torna o método adequado a implementações paralelas, já que iterações podem ser distribuídas entre processadores sem coordenação.

1.5 Variações da construção e do parâmetro α

Usar um único valor fixo de α costuma atrapalhar a busca por soluções de alta qualidade, que poderiam ser encontradas com outro valor. Há três alternativas usuais. A primeira é manter α fixo. A segunda é sortear α de maneira uniforme no intervalo $[0, 1]$ a cada iteração, o que introduz diversidade sem custo de ajuste. A terceira é o GRASP Reativo, no qual α é auto-ajustado ao longo da execução, com base na qualidade das soluções obtidas anteriormente com cada valor. Relatos na literatura indicam que o GRASP Reativo tende a superar a versão básica, e que estratégias que variam α tendem a ser mais efetivas do que um valor fixo único.

Também existem esquemas de construção com complexidade de pior caso menor do que a do Algoritmo 1. No esquema *random plus greedy*, aplica-se aleatoriedade nos primeiros p passos, produzindo uma solução parcial aleatória, e o restante é completado de forma gulosa. No esquema *sampled greedy*, a cada passo amostram-se $\min\{p, |C|\}$ elementos do conjunto de candidatos, avalia-se apenas essa amostra pela função gulosa e escolhe-se o de melhor valor. Em ambos, o parâmetro p controla o balanço entre ganância e aleatoriedade: amostras pequenas produzem soluções mais aleatórias, amostras grandes produzem soluções mais gulosas.

1.6 Intensificação e path-relinking

O GRASP clássico é o que foi descrito até aqui: construção mais busca local, repetidas, guardando a melhor solução. A partir dele foram propostas extensões de intensificação, sendo a mais conhecida o *path-relinking*. Nessa técnica, exploram-se trajetórias no espaço de soluções ligando uma solução corrente a soluções de elite guardadas em um conjunto, na tentativa de incorporar atributos das soluções de boa qualidade. O path-relinking adiciona memória ao procedimento e costuma melhorar tempo e qualidade, mas não faz parte do GRASP na forma original do template. Neste trabalho ele não é usado na versão clássica aqui documentada; a combinação de GRASP com outras estratégias é tratada por um método separado no projeto.

1.7 Boas práticas relevantes

Alguns pontos práticos do método são diretamente aproveitados na Seção 2. Bons pontos de partida costumam levar a melhores soluções finais e reduzir o tempo de busca local. A busca local pode ser bastante acelerada com estruturas de dados adequadas e com listas de candidatos que armazenam ou estimam valores de movimento, evitando reavaliações completas. A varredura por primeira melhora tende a ser preferível pelo menor tempo e menor risco de convergência prematura. Por fim, não existe metaheurística universal: a estrutura de cada problema deve ser explorada na construção do

algoritmo, e conhecimento sobre o problema deve ser usado para desenhar funções de avaliação e vizinhanças eficientes.

2 GRASP no projeto

Esta seção relaciona o método descrito na Seção 1 com a implementação feita no projeto. A cada elemento da teoria corresponde uma decisão concreta de modelagem ou de código, e essas correspondências são explicitadas ao longo do texto. A implementação está no módulo `metaheuristics/methods/grasp.py` e reutiliza a infraestrutura comum de avaliação e a busca local já usada pelo método de *Iterated Local Search* (ILS).

2.1 O problema de otimização

O projeto calcula, para cada hexágono H3 de uma região, um conjunto de indicadores de caminhabilidade e, a partir deles, o Índice de Qualidade Caminhável (IQC). A área de estudo é discretizada no conjunto de hexágonos $\mathcal{H} = \{h_1, h_2, \dots, h_{|\mathcal{H}|}\}$, e as dimensões formam o conjunto $\mathcal{K}$, com $|\mathcal{K}| = 11$: S_saude, S_educacao, S_abastecimento, S_lazer, S_servicos, I_seguranca, A_vegetacao, A_agua, C_conectividade, T_transporte e U_urbanidade. O IQC de um hexágono é uma agregação ponderada dos indicadores, com pesos w_j obtidos pelo método CRITIC.

A otimização consiste em alocar um orçamento fixo B de pontos de interesse (POIs) na região, com o objetivo de maximizar a soma do IQC sobre todos os hexágonos. Os POIs só podem ser inseridos no subconjunto de dimensões instaláveis $\mathcal{K}_{POI} \subset \mathcal{K}$, com $|\mathcal{K}_{POI}| = 7$: S_saude, S_educacao, S_abastecimento, S_lazer, S_servicos, T_transporte e U_urbanidade. Um POI inserido em um hexágono de origem não afeta apenas aquele hexágono: seu efeito se propaga para os hexágonos de destino segundo um modelo de impacto temporal. Cada par origem-destino tem um peso $\phi(t_{i',i})$ derivado do tempo de caminhada $t_{i',i}$ entre eles, por um decaimento em cosseno,

$$\phi(t) = \begin{cases} \frac{1 + \cos(\pi t/t_{max})}{2}, & 0 \leq t \leq t_{max}, \\ 0, & \text{caso contrário,} \end{cases} \quad (2)$$

com $t_{max} = 20$ minutos (ϕ corresponde à coluna `alpha_20` dos dados). Assim, inserir um POI perto de muitos destinos alcançáveis rende mais do que inseri-lo em um ponto isolado. O orçamento adotado é $B = 100$ POIs, e a soma do IQC sem nenhuma alocação é a linha de base (*baseline*) usada para medir o ganho.

2.2 Representação da solução e conjunto-base

A correspondência com a teoria começa na definição do conjunto-base. Um elemento do conjunto-base E é a inserção de uma unidade de POI em um par (hexágono de origem h_i , dimensão j). Uma solução é a matriz $\Delta\mathbf{X}$, de dimensão $|\mathcal{H}| \times |\mathcal{K}_{POI}|$, em que cada entrada $\Delta x_{i,j} \in \mathbb{Z}_{\geq 0}$ é a quantidade de POIs da dimensão j alocada no hexágono h_i ; as entradas somam o orçamento, $\sum_i \sum_j \Delta x_{i,j} = B$, e são não nulas apenas nas linhas correspondentes a hexágonos de origem elegíveis. Cada unidade alocada é, portanto, um elemento incorporado à solução.

Sob essa representação, o orçamento coincide com o número de passos da construção: como cada passo insere uma unidade de POI, toda solução construída tem exatamente $B = 100$ POIs. Essa escolha mantém a solução dentro do mesmo espaço de busca usado pelo ILS e preserva a mesma invariante de conservação de orçamento, o que é necessário para que a comparação entre os dois métodos seja feita sobre o mesmo conjunto de soluções viáveis.

2.3 Glossário de variáveis

Para acompanhar os algoritmos das próximas subseções, a Tabela 1 reúne os símbolos matemáticos, os nomes usados no código e o significado de cada variável.

Tabela 1: Variáveis usadas na formulação e no código.

Símbolo	Nome no código	Significado
E	—	conjunto-base; pares (origem h_i , dimensão j)
S, S^*	—	solução corrente e melhor solução
$F(\Delta\mathbf{X})$	<code>objective_value</code>	aptidão; $\overline{\text{IQC}}$, soma do IQC da solução
B	<code>budget</code>	orçamento; número de POIs a alocar (100)
$\mathcal{H}, \mathcal{H} $	<code>h3_ids, n_hex</code>	conjunto de hexágonos e sua quantidade
$\mathcal{K}_{POI}$	<code>candidate_dimensions, n_dims</code>	dimensões instaláveis de POI (sete)
$\mathcal{K}$	<code>indicator_columns</code>	os onze indicadores usados no IQC
—	<code>candidate_to_indicator_indices (cols)</code>	índices da dimensão de POI, o índice da coluna de indicador correspondente
$S_{i,j}$	<code>baseline_matrix</code>	oferta de base ($ \mathcal{H} \times 11$)
$S_{i,j}^s(\Delta\mathbf{X})$	<code>final_matrix</code>	oferta após aplicar o impacto da alocação
$\Delta x_{i,j}, \Delta\mathbf{X}$	<code>candidate_matrix (cand)</code>	solução; quantidade de POIs no hexágono h_i , dimensão j

Símbolo	Nome no código	Significado
$\phi(t_{i',i})$	<code>alpha_values</code>	peso de impacto do par origem $h_{i'}$ / destino h_i
—	<code>source_indices,</code> <code>target_indices</code>	índices de origem e destino de cada par de impacto
—	<code>allowed_source_rows</code>	linhas (hexágonos) elegíveis a receber POI
$\text{alcance}(i)$	<code>reach_row</code>	soma dos pesos de impacto que saem de h_i
w_j	<code>base_weights</code>	peso CRITIC da dimensão j
$\min_j, \max_j$	<code>col_min, col_max</code>	mínimo e máximo da coluna j na matriz corrente
—	<code>range_safe</code>	faixa $\max_j - \min_j$, com piso para evitar divisão por zero
—	<code>col_factor</code>	fator de coluna $w_j / (\max_j - \min_j)$
—	<code>score_grid</code>	escore surrogate por (origem elegível, dimensão)
α	<code>alpha</code>	parâmetro de limiar da RCL
—	<code>threshold</code>	limiar de corte da RCL
—	<code>rcl_positions,</code> <code>rcl_size</code>	elementos da RCL e seu tamanho
—	<code>random_generator (rng)</code>	gerador pseudoaleatório da semente
—	<code>evaluations</code>	número de avaliações reais da função objetivo
—	<code>iterations</code>	número de iterações GRASP na execução
—	<code>no_improve_streak</code>	iteraões seguidas sem melhorar a melhor solução
—	<code>max_evaluations</code>	teto de avaliações (30 000)
—	<code>max_iterations</code>	teto de iterações (500)
—	<code>max_no_improve</code>	teto de iterações sem melhora (80)

2.4 A função objetivo compartilhada

Todos os métodos do projeto avaliam uma solução pela mesma função objetivo, implementada em `metaheuristics/core/evaluation.py`. A avaliação tem duas etapas: construir a matriz final de indicadores a partir da linha de base e do impacto da alocação, e depois recalculando os pesos CRITIC e o IQC sobre essa matriz. A Listagem 1 mostra as duas etapas.

Listing 1: Construção da matriz final e avaliação do objetivo.

```
1 def build_final_indicator_matrix_nd(candidate_matrix, state):
```



```

2   n_hex = len(state.h3_ids)
3   n_dims = len(state.candidate_dimensions)
4   allocation_rows = candidate_matrix[state.source_indices]
5   weighted_rows = allocation_rows * state.alpha_values[:, None]
6   delta_by_target = np.zeros((n_hex, n_dims))
7   np.add.at(delta_by_target, state.target_indices, weighted_rows)
8   final_matrix = state.baseline_matrix.copy()
9   final_matrix[:, state.candidate_to_indicator_indices] +=
    delta_by_target
10  return final_matrix
11
12 def objective_function(final_indicator_matrix):
13     critic_weights = _compute_critic_weights_numpy(
14         final_indicator_matrix)
15     iqc_values = _compute_iqc_numpy(final_indicator_matrix,
16                                     critic_weights)
17     objective_value = float(iqc_values.sum())
18     return {"objective_value": objective_value, ...}

```

Explicando a Listagem 1 linha a linha. A linha 4 seleciona, para cada par de impacto, a linha da alocação correspondente ao hexágono de origem daquele par. A linha 5 multiplica cada uma dessas linhas pelo peso ϕ do par, o que dá a contribuição que sai de cada origem para o respectivo destino. A linha 6 cria uma matriz de deltas por destino, zerada. A linha 7 soma, em cada destino, as contribuições vindas de todas as origens que o alcançam; a função `np.add.at` faz essa soma acumulando corretamente quando vários pares apontam para o mesmo destino. A linha 8 copia a matriz de base, e a linha 9 soma os deltas nas colunas de indicador que correspondem às dimensões de POI. O resultado é a Equação (3), em que $\mathcal{N}_i$ é o conjunto de origens que alcançam o hexágono h_i dentro de t_{max} :

$$S_{i,j}^s(\Delta \mathbf{X}) = S_{i,j} + \sum_{h_{i'} \in \mathcal{N}_i} \phi(t_{i',i}) \Delta x_{i',j}. \quad (3)$$

Na segunda função, a linha 13 recalcula os pesos CRITIC sobre a matriz final, a linha 14 recalcula o IQC de cada hexágono e a linha 15 soma esses valores para obter o objetivo.

A normalização usada no IQC é min-max por coluna,

$$\hat{S}_{i,j}^s = \begin{cases} \frac{S_{i,j}^s - \min_j}{\max_j - \min_j}, & \max_j > \min_j, \\ 0,5, & \max_j = \min_j, \end{cases} \quad (4)$$

e o IQC de um hexágono é a soma ponderada $\text{IQC}_i = \sum_{j \in \mathcal{K}} w_j \hat{S}_{i,j}^s$, arredondada a quatro casas decimais. A aptidão de uma solução é o IQC global,

$$F(\Delta \mathbf{X}) = \overline{\text{IQC}}(\Delta \mathbf{X}) = \sum_{i \in \mathcal{H}} \text{IQC}_i, \quad (5)$$

a ser maximizada; nas descrições dos algoritmos, $F(S)$ denota a aptidão da solução corrente $S = \Delta \mathbf{X}$. Os pesos w_j vêm do CRITIC: para cada coluna calcula-se o desvio padrão da coluna normalizada, σ_j , e uma medida de conflito $\sum_{j'} (1 - |\rho_{j,j'}|)$, com $\rho_{j,j'}$ a correlação entre as colunas j e j' ; o produto $\sigma_j \sum_{j'} (1 - |\rho_{j,j'}|)$ é normalizado para somar um.

Como a função é de maximização, a RCL correspondente à Equação (1) passa a selecionar os elementos de maior ganho. Se $g(e)$ é o ganho de incorporar o elemento e , e $g^{\max}$ e $g^{\min}$ são o maior e o menor ganhos entre os candidatos, a RCL usada é

$$\text{RCL} = \{ e \in C : g(e) \geq g^{\max} - \alpha (g^{\max} - g^{\min}) \}. \quad (6)$$

2.5 Por que a função objetivo não tem custo incremental barato

A teoria do Algoritmo 1 pressupõe que se possa avaliar o custo (ou ganho) incremental de cada candidato a cada passo. No caso do IQC isso é caro. O problema está no CRITIC e na normalização: os pesos w_j dependem do desvio padrão e das correlações de todas as colunas, e a normalização depende do mínimo e do máximo de cada coluna. Inserir uma única unidade de POI altera muitos hexágonos de destino, o que muda as estatísticas das colunas e, com elas, os pesos. O efeito marginal de um elemento não é independente do restante da solução, ou seja, a função objetivo é não separável.

Na prática, calcular o ganho exato de cada candidato exigiria uma avaliação completa da Equação (5) por candidato, a cada passo da construção. O número de candidatos é da ordem de (número de hexágonos de origem) $\times 7$ dimensões, o que dá aproximadamente mil e duzentos candidatos em uma região típica. Uma construção gulosa exata custaria, então, cerca de $100 \times 1200 = 120\,000$ avaliações para montar uma única solução, mais do que o orçamento total de avaliações de uma execução inteira do ILS. Isso inviabiliza a construção gulosa exata para um método genuinamente multi-start, que precisa montar muitas soluções.

2.6 Pré-compilação do estado e do escore surrogate

Antes do laço, o método precompila os valores que não mudam durante a execução. A Listagem 2 mostra esse passo.

Listing 2: Pré-compilação do escore surrogate.

```

1 def _build_construction_surrogate(state):
2     baseline = state.baseline_matrix
3     n_hex = baseline.shape[0]
4     full_weights = _compute_critic_weights_numpy(baseline)
5     base_weights = full_weights[state.candidate_to_indicator_indices]
6     reach_row = np.zeros(n_hex)
7     np.add.at(reach_row, state.source_indices, state.alpha_values)

```

```

8     order = np.argsort(state.source_indices, kind="stable")
9     sorted_sources = state.source_indices[order]
10    sorted_targets = state.target_indices[order]
11    sorted_alphas = state.alpha_values[order]
12    source_targets, source_alphas = {}, {}
13    unique_sources, start = np.unique(sorted_sources, return_index=True)
14    bounds = list(start) + [sorted_sources.size]
15    for i, s in enumerate(unique_sources):
16        lo, hi = bounds[i], bounds[i + 1]
17        source_targets[int(s)] = sorted_targets[lo:hi]
18        source_alphas[int(s)] = sorted_alphas[lo:hi]
19    return base_weights, reach_row, source_targets, source_alphas

```

Linha a linha da Listagem 2. A linha 4 calcula os pesos CRITIC da solução vazia, ou seja, sobre a matriz de base; esse cálculo é numérico e não conta como avaliação da função objetivo. A linha 5 seleciona, entre esses pesos, apenas os das sete dimensões de POI, guardando-os em **base_weights**. As linhas 6 e 7 calculam o alcance de cada hexágono, somando os pesos ϕ de todos os pares de impacto que partem daquele hexágono; **np.add.at** acumula por índice de origem. As linhas 8 a 11 ordenam os pares de impacto por origem, para poder agrupá-los. As linhas 12 a 18 constroem, para cada origem, a lista dos seus destinos e a lista dos pesos correspondentes, guardadas em **source_targets** e **source_alphas**. Esses dois dicionários permitem, durante a construção, atualizar a matriz final de forma incremental sem varrer todos os pares. A linha 19 devolve os quatro objetos precompilados. Como toda construção começa da solução vazia, **base_weights** e **reach_row** são constantes ao longo de toda a execução e por isso são calculados uma única vez.

2.7 Fase de construção linha a linha

A construção gulosa e aleatorizada está na Listagem 3. Ela é a versão para maximização do Algoritmo 1, com a diferença de que a função gulosa é o escore surrogate, não o ganho exato.

Listing 3: Construção gulosa e aleatorizada com escore surrogate.

```

1  def _greedy_randomized_construction(state, sur, rng, allowed_rows,
2      budget, alpha, eps):
3      n_hex = state.baseline_matrix.shape[0]
4      n_dims = len(state.candidate_dimensions)
5      cols = sur.candidate_to_indicator_indices
6      final = state.baseline_matrix.copy()
7      cand = np.zeros((n_hex, n_dims))
8      reach_allowed = sur.reach_row[allowed_rows]
9      rcl_sizes = []
10     for _ in range(budget):
11         sub = final[:, cols]
12         col_min = sub.min(axis=0)
13         col_max = sub.max(axis=0)

```

```

14     col_range = col_max - col_min
15     range_safe = np.where(col_range > eps, col_range, eps)
16     col_factor = sur.base_weights / range_safe
17     score = reach_allowed[:, None] * col_factor[None, :]
18     smax = score.max(); smin = score.min()
19     threshold = smax - alpha * (smax - smin)
20     rcl = np.argwhere(score >= threshold - 1e-15)
21     rcl_sizes.append(rcl.shape[0])
22     k = int(rng.integers(0, rcl.shape[0]))
23     local_row, d = int(rcl[k, 0]), int(rcl[k, 1])
24     s = int(allowed_rows[local_row])
25     cand[s, d] += 1.0
26     t = sur.source_targets.get(s)
27     if t is not None and t.size:
28         final[t, int(cols[d])] += sur.source_alphas[s]
29     return cand, float(np.mean(rcl_sizes))

```

Linha a linha da Listagem 3. As linhas 6 e 7 iniciam a matriz final como cópia da base e a solução `cand` como matriz de zeros. A linha 8 recolhe o alcance apenas das linhas elegíveis, que serão as linhas da grade de escores. O laço da linha 10 repete-se `budget` vezes, uma inserção de POI por passo. As linhas 11 a 14 extraem as colunas de indicador correspondentes às dimensões de POI e calculam o mínimo, o máximo e a faixa de cada uma na matriz final corrente. A linha 15 aplica um piso na faixa para evitar divisão por zero quando a coluna é constante. A linha 16 calcula o fator de coluna, $w_j/(\max_j - \min_j)$. A linha 17 forma a grade de escores como o produto externo entre o alcance das linhas elegíveis e o fator de coluna, o que corresponde à Equação (7). As linhas 18 e 19 obtêm o maior e o menor escore e calculam o limiar da RCL segundo a Equação (6). A linha 20 seleciona as posições (origem, dimensão) cujo escore está acima do limiar, e a linha 21 registra o tamanho dessa RCL. A linha 22 sorteia uma posição da RCL usando o gerador da semente, e as linhas 23 e 24 traduzem essa posição para a linha de origem real e a dimensão escolhida. A linha 25 insere uma unidade de POI na solução. As linhas 26 a 28 atualizam a matriz final de forma incremental: para a origem escolhida, somam o peso ϕ de cada destino na coluna de indicador da dimensão inserida. A linha 29 devolve a solução construída e o tamanho médio da RCL.

O escore da linha 17 é a estimativa que substitui o ganho exato. Mantendo pesos e faixas fixos, o ganho marginal de inserir uma unidade em (origem $h_{i'}$, dimensão j) sobre o IQC global é aproximadamente

$$\Delta F(i', j) \approx \frac{w_j}{\max_j - \min_j} \sum_{h_i \in \mathcal{N}_{i'}} \phi(t_{i', i}) = \frac{w_j}{\max_j - \min_j} \text{alcance}(i'). \quad (7)$$

A dedução vem de observar que aumentar o indicador j em um destino h_i por $\phi(t_{i', i})$ aumenta o IQC daquele destino em $w_j \cdot \phi(t_{i', i})/(\max_j - \min_j)$; somando sobre os destinos alcançáveis a partir de $h_{i'}$ chega-se à Equação (7). A estimativa é separável em um fator de linha, $\text{alcance}(i')$, e um fator de

coluna, $w_j/(\max_j - \min_j)$, o que permite montar a grade inteira em uma operação de matriz, sem nenhuma avaliação da função objetivo. Os pesos w_j ficam fixos, mas as faixas são recalculadas a cada passo (linhas 12 a 16); conforme uma dimensão é preenchida, sua faixa cresce e seu fator de coluna cai, o que produz retornos decrescentes e espalha as inserções entre as dimensões.

O parâmetro α da linha 19 é sorteado de maneira uniforme em $[0, 1]$ a cada iteração do GRASP, conforme a segunda alternativa da Seção 1. Optou-se por essa estratégia, e não por um α fixo, para introduzir diversidade sem depender de ajuste. O GRASP Reativo foi considerado, mas não adotado nesta versão, por acrescentar parâmetros e código sem necessidade para o objetivo da comparação.

2.8 Fase de busca local linha a linha

A busca local não foi reescrita: a implementação importa e usa o mesmo operador do ILS. Como os dois métodos passam a compartilhar exatamente o mesmo operador, a diferença entre eles fica isolada na construção e no caráter multi-start do GRASP, contra a perturbação no ILS. O operador tem duas partes: o sorteio de um movimento de realocação e o laço de primeira melhora.

Listing 4: Sorteio de um movimento de realocação viável.

```

1 def _sample_feasible_relocation_move(cand, rng, allowed_rows,
2                                     preserve_dimension=True):
3     src_positions = np.argwhere(cand[allowed_rows, :] > 0)
4     if src_positions.size == 0:
5         return None
6     n_cols = cand.shape[1]
7     n_rows = allowed_rows.shape[0]
8     for _ in range(12):
9         i = int(rng.integers(0, len(src_positions)))
10        src_row_local, src_col = src_positions[i]
11        src_row = int(allowed_rows[src_row_local])
12        tgt_row = int(allowed_rows[rng.integers(0, n_rows)])
13        tgt_col = int(src_col) if preserve_dimension else int(rng.
14        integers(0, n_cols))
15        if src_row == tgt_row and src_col == tgt_col:
16            continue
17        return src_row, src_col, tgt_row, tgt_col
18    return None

```

Na Listagem 4, a linha 3 lista as posições da solução, restritas às linhas elegíveis, que já têm ao menos um POI; são as origens possíveis do movimento. As linhas 4 e 5 abortam se não houver nenhuma. O laço da linha 8 tenta até doze vezes montar um movimento válido. A linha 9 sorteia uma posição de origem, e as linhas 10 e 11 recuperam a linha e a coluna dessa origem. A linha 12 sorteia a linha de destino entre as elegíveis. A linha 13 define

a coluna de destino: quando `preserve_dimension` é verdadeiro, mantém-se a mesma dimensão da origem, o que é o modo usado aqui. As linhas 14 e 15 descartam o movimento nulo, em que origem e destino coincidem. A linha 16 devolve o movimento como a quádrupla (linha de origem, coluna de origem, linha de destino, coluna de destino).

Listing 5: Busca local por primeira melhora.

```

1 def _run_local_search_first_improvement(cand, state, rng, value,
2     max_steps, neighbor_sample, allowed_rows, eps,
3     preserve_dimension=True):
4     accepted = 0
5     for _ in range(max_steps):
6         improved = False
7         for _ in range(neighbor_sample):
8             move = _sample_feasible_relocation_move(
9                 cand, rng, allowed_rows, preserve_dimension)
10            if move is None:
11                break
12            sr, sc, tr, tc = move
13            cand[sr, sc] -= 1.0
14            cand[tr, tc] += 1.0
15            trial = objective_function(
16                build_final_indicator_matrix_nd(cand, state))["
objective_value"]
17            if trial > value + eps:
18                value = trial
19                accepted += 1
20                improved = True
21                break
22            cand[tr, tc] -= 1.0
23            cand[sr, sc] += 1.0
24        if not improved:
25            break
26    return value, accepted

```

Na Listagem 5, o laço externo da linha 5 executa até `max_steps` passos de melhora. A cada passo, o laço interno da linha 7 sorteia até `neighbor_sample` vizinhos. A linha 8 obtém um movimento; se não houver movimento viável, a linha 11 interrompe. As linhas 13 e 14 aplicam o movimento na solução, tirando uma unidade da origem e pondo uma no destino. As linhas 15 e 16 avaliam a solução alterada com a função objetivo verdadeira, o que conta como uma avaliação. A linha 17 testa se houve melhora acima de uma tolerância. Se sim, as linhas 18 a 21 aceitam o movimento, registram a aceitação e saem do laço interno, o que caracteriza a estratégia de primeira melhora: aceita-se o primeiro vizinho que melhore, sem varrer os demais. Se não houve melhora, as linhas 22 e 23 desfazem o movimento, deixando a solução como estava. Quando um passo inteiro do laço externo não encontra nenhuma melhora, a linha 25 encerra a busca. A vizinhança completa é grande, por isso o operador usa amostragem em vez de examinar todos os vizinhos; essa

Algorithm 3 GRASP implementado, por semente (maximização de $\overline{IQC}$)

```
1: procedure GRASP-SEMENTE(mente)
2:   inicialize o gerador pseudoaleatório com mente
3:    $F^* \leftarrow -\infty$ ;   $avaliacoes \leftarrow 0$ ;   $iteracoes \leftarrow 0$ ;   $sem\_melhora \leftarrow 0$ 
4:   pré-compile pesos base e alcance por origem ▷ Listagem 2
5:   while  $avaliacoes < 30000$  e  $iteracoes < 500$  e  $sem\_melhora < 80$  do
6:      $iteracoes \leftarrow iteracoes + 1$ 
7:      $\alpha \leftarrow$  valor uniforme em  $[0, 1]$ 
8:      $S \leftarrow$  ConstruaGulosaAleatorizada( $\alpha$ ) ▷ Listagem 3; sem
       avaliações
9:     avalie  $F(S)$ ;   $avaliacoes \leftarrow avaliacoes + 1$ 
10:     $S \leftarrow$  BuscaLocalPrimeiraMelhora( $S$ ) ▷ Listagem 5; acumula
      avaliações
11:    if  $F(S) > F^*$  then
12:       $F^* \leftarrow F(S)$ ;   $S^* \leftarrow S$ ;   $sem\_melhora \leftarrow 0$ ;  registre  $\alpha$ 
13:    else
14:       $sem\_melhora \leftarrow sem\_melhora + 1$ 
15:    end if
16:    registre a trajetória da iteração
17:  end while
18:  return  $S^*$ ,  $F^*$  e as métricas da execução
19: end procedure
```

é a estratégia de vizinhança restrita descrita na teoria.

2.9 O laço GRASP por semente

O laço por semente segue o template do Algoritmo 2, com o critério de parada por orçamento de avaliações. O Algoritmo 3 descreve o procedimento implementado.

Linha a linha do Algoritmo 3. A linha 2 inicializa o gerador da semente, que será usado em toda a execução. A linha 3 zera os contadores e coloca o melhor valor como menos infinito, já que é maximização. A linha 4 precompila os valores fixos. O laço da linha 5 continua enquanto nenhum dos três limites for atingido. A linha 7 sorteia α para a iteração. A linha 8 constrói uma solução, sem gastar avaliações. A linha 9 avalia a solução construída, o que gasta uma avaliação. A linha 10 aplica a busca local, que gasta uma avaliação por vizinho testado. As linhas 11 a 15 atualizam a melhor solução da execução: quando a solução após a busca local supera o melhor valor, ela é guardada, o contador de iterações sem melhora é zerado e o α da iteração é registrado; caso contrário, o contador de iterações sem melhora é incrementado. A linha 16 grava a trajetória, com os valores da iteração. A linha 18 devolve a melhor solução e as métricas.

Como o GRASP clássico não guarda estado entre iterações, não há critério de aceitação entre uma iteração e a seguinte, ao contrário do ILS: cada construção parte do zero. O que se mantém é a melhor solução global da execução.

2.10 Fluxo de execução

A Figura 1 resume o fluxo de uma execução por semente, e a Figura 2 mostra como o orçamento de avaliações se distribui dentro de uma iteração. Os dois diagramas condensam o que as listagens anteriores detalham: a construção não consome avaliações; a avaliação da solução construída consome uma; e a busca local consome uma avaliação por vizinho testado, até o limite de $25 \times 64 = 1\,600$.

2.11 Sementes, reprodutibilidade e paralelismo

As sementes são lidas do arquivo `seeds.txt`, que contém 100 inteiros. Cada semente corresponde a uma execução completa e independente do GRASP. A semente cumpre o papel do parâmetro *semente* do Algoritmo 2: ela inicializa o gerador pseudoaleatório, que governa tanto o sorteio dentro da RCL na construção (linha 22 da Listagem 3) quanto a amostragem de vizinhos na busca local (Listagem 4). Com isso, cada execução é reproduzível, e a execução da semente i do GRASP fica pareada com a execução da mesma semente i do ILS, o que permite comparações par a par.

Como as iterações do GRASP são independentes, e como as execuções por semente também são independentes entre si, elas são distribuídas entre processos paralelos com `ProcessPoolExecutor`. Isso corresponde à observação da Seção 1 de que o GRASP é adequado a implementações paralelas pela independência de suas iterações. O resultado por semente não depende do paralelismo, porque cada execução usa seu próprio gerador determinístico.

2.12 Contabilização de avaliações e comparabilidade com o ILS

A comparação entre GRASP e ILS foi desenhada para ser feita sob o mesmo esforço computacional. A medida de esforço é o número de avaliações da função objetivo, e os dois métodos são limitados pelo mesmo teto de 30 000 avaliações por semente. Para que essa contabilização seja justa, a construção do GRASP não gasta avaliações: o escore surrogate da Equação (7) é um cálculo vetorizado que não chama a função objetivo. Só contam como avaliações a da solução construída (linha 9 do Algoritmo 3) e as da busca local. Como a construção é barata, cada iteração custa poucas avaliações, e o teto permite muitas iterações por semente, o que preserva o caráter multi-start. Os limites de 500 iterações e de 80 iterações sem melhora funcionam como salvaguardas e são os mesmos do ILS.

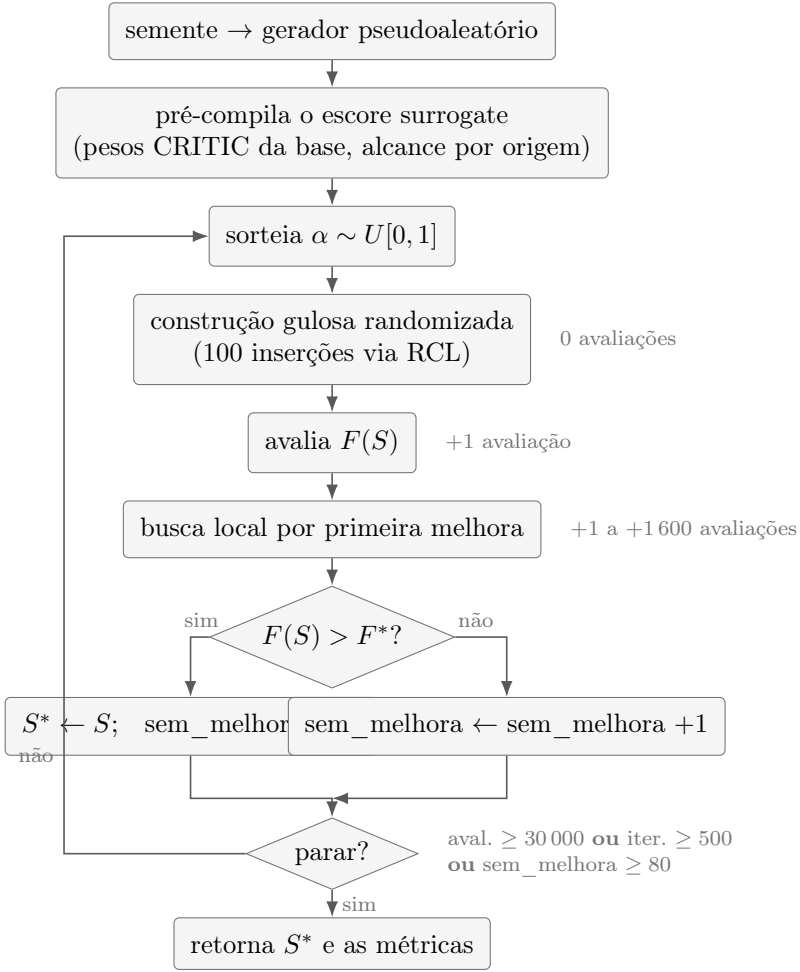


Figura 1: Fluxo de uma execução GRASP por semente, como implementado. Cada volta do laço é uma iteração independente: sorteio de α , construção, avaliação, busca local e atualização da melhor solução.

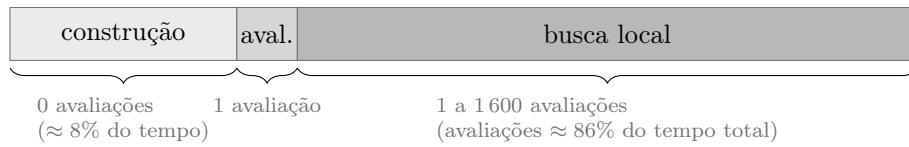


Figura 2: Anatomia de uma iteração GRASP sob a ótica do orçamento. A construção usa apenas o escore surrogate, sem chamar a função objetivo; o custo em avaliações concentra-se na busca local. As frações de tempo vêm dos registros de perfil das execuções.

2.13 Exemplo de execução passo a passo

Esta subseção executa o método sobre uma instância pequena, para mostrar os números em cada etapa. A instância tem quatro hexágonos, h_0 a h_3 , e duas dimensões de POI, **S_saude** e **S_lazer**. Há ainda um terceiro indicador não sujeito a POI, **I_seguranca**, que participa do IQC mas não recebe alocação. Apenas h_0 e h_1 são elegíveis a receber POI. O orçamento é de três POIs. A Tabela 2 mostra a matriz de indicadores de base.

Tabela 2: Matriz de oferta de base $S_{i,j}$ do exemplo.

Hexágono	S_saude	S_lazer	I_seguranca
h_0	0,20	0,10	0,50
h_1	0,10	0,30	0,40
h_2	0,00	0,00	0,60
h_3	0,30	0,20	0,50

Os pares de impacto são: de h_0 para h_0 com peso 1,00, para h_2 com 0,50 e para h_3 com 0,25; de h_1 para h_1 com 1,00 e para h_2 com 0,50. Somando os pesos que saem de cada origem, o alcance é $\text{alcance}(h_0) = 1,75$ e $\text{alcance}(h_1) = 1,50$. Os pesos CRITIC calculados sobre a matriz de base são 0,4877 para **S_saude**, 0,2474 para **S_lazer** e 0,2649 para **I_seguranca**; como dimensões de POI, ficam $w_{\text{saude}} = 0,4877$ e $w_{\text{lazer}} = 0,2474$.

O objetivo da solução vazia é a linha de base. Os IQC por hexágono são 0,5400, 0,4100, 0,2649 e 0,7851, cuja soma é $F = 2,0000$.

A seguir, uma construção com $\alpha = 0,5$. As Tabelas 3 resumem os três passos. Em cada passo, a grade de escores tem duas linhas (h_0 e h_1) e duas colunas (**S_saude** e **S_lazer**), e o limiar corta a RCL.

Tabela 3: Passos da construção no exemplo ($\alpha = 0,5$).

Passo	Faixas (max – min) [S_saude, S_lazer]	Escore h_0 [saude, lazer]	Escore h_1 [saude, lazer]	Limiar	Escolha
1	[0,30, 0,30]	[2,845, 1,443]	[2,438, 1,237]	2,041	h_1 , S_saude
2	[0,90, 0,30]	[0,948, 1,443]	[0,813, 1,237]	1,128	h_1 , S_lazer
3	[0,90, 1,20]	[0,948, 0,361]	[0,813, 0,309]	0,629	h_1 , S_saude

No passo 1, todas as colunas de POI têm faixa 0,30. Os fatores de coluna são $0,4877/0,30 = 1,626$ para **S_saude** e $0,2474/0,30 = 0,825$ para **S_lazer**. Multiplicando pelos alcances, o maior escore é $1,75 \times 1,626 = 2,845$, em $(h_0, \text{S_saude})$, e o menor é 1,237, em $(h_1, \text{S_lazer})$. O limiar

$2,845 - 0,5(2,845 - 1,237) = 2,041$ deixa na RCL as duas posições da coluna **S_saude**: $(h_0, \mathbf{S_saude})$ com 2,845 e $(h_1, \mathbf{S_saude})$ com 2,438. O sorteio, com a semente do exemplo, escolhe $(h_1, \mathbf{S_saude})$. Note que a escolha gulosa pura teria pego $(h_0, \mathbf{S_saude})$, de escore maior; foi a aleatorização da RCL que levou a solução para h_1 .

No passo 2, inserir um POI de **S_saude** em h_1 elevou os valores dessa coluna nos destinos de h_1 , o que aumentou a faixa de **S_saude** de 0,30 para 0,90 e reduziu seu fator de coluna de 1,626 para 0,542. Com isso, a coluna **S_lazer** passou a ter os maiores escores, e a RCL ficou com as duas posições de **S_lazer**. O sorteio escolhe $(h_1, \mathbf{S_lazer})$. Esse é o efeito de retornos decrescentes: preencher uma dimensão reduz o interesse por ela nos passos seguintes.

No passo 3, agora a faixa de **S_lazer** subiu para 1,20, e **S_saude** volta a ter os maiores escores. A RCL fica com as duas posições de **S_saude**, e o sorteio escolhe de novo $(h_1, \mathbf{S_saude})$. A solução construída aloca dois POIs de **S_saude** e um de **S_lazer**, todos em h_1 .

Avaliando essa solução com a função objetivo verdadeira, a matriz final tem h_1 com **S_saude** igual a 2,1 e **S_lazer** igual a 1,3, e h_2 com **S_saude** igual a 1,0 e **S_lazer** igual a 0,5, por causa da propagação de h_1 para os destinos h_1 e h_2 . Os IQC por hexágono passam a 0,2341, 0,5318, 0,6713 e 0,2694, cuja soma é $F = 1,7066$.

Vale observar que a soma caiu em relação à linha de base, de 2,0000 para 1,7066. Isso acontece porque concentrar os POIs em um único hexágono eleva muito o máximo das colunas **S_saude** e **S_lazer**; como a normalização é min-max, esse máximo alto empurra para baixo os valores normalizados de todos os outros hexágonos, e o ganho no hexágono alocado não compensa a perda nos demais. Esse resultado mostra duas coisas. Primeiro, o escore surrogate é apenas uma estimativa para ordenar candidatos, e não a medida real de qualidade; a solução construída pode até piorar o objetivo. Segundo, é a busca local, avaliada pela função verdadeira, que corrige esse tipo de situação.

A busca local parte da solução construída, com dois POIs de **S_saude** e um de **S_lazer** em h_1 . Um dos movimentos possíveis é transferir uma unidade de **S_saude** de h_1 para h_0 , mantendo a dimensão. Avaliando, a soma do IQC passa de 1,7066 para 2,0465, um ganho de +0,3399; como é o primeiro movimento testado que melhora, ele é aceito. Outro movimento, transferir a unidade de **S_lazer** de h_1 para h_0 , levaria a soma de 1,7066 para 1,6591, uma piora de -0,0475, e seria desfeito. Depois do movimento aceito, a solução já supera a linha de base, com 2,0465 contra 2,0000, porque distribuir os POIs entre h_0 e h_1 reduz a concentração e melhora o equilíbrio das colunas. A busca local continuaria testando outros movimentos até não encontrar melhora, e a melhor solução da iteração seria comparada com a melhor da execução.

2.14 Artefatos de saída e registros

A saída do GRASP reproduz a organização usada pelo ILS, trocando apenas a raiz `results/ils/` por `results/grasp/`. Para cada execução, os arquivos ficam sob `results/grasp/<localidade>/<perfil>/<dataset>/<run_id>/`, com três partes. Em `config/` guarda-se `experiment_config.json`, com os parâmetros da execução. Em `summary/` guardam-se `runs_summary.csv` (uma linha por semente), `summary_stats.json` (estatísticas agregadas sobre as sementes), `global_best_allocation.csv` (a melhor alocação encontrada) e `method_result.json`. Em `seed_runs/seed_XXXXXX/` guardam-se, por semente, `trajectory.csv` (uma linha por iteração), `best_allocation.csv` e `run_metrics.json`.

As colunas usadas na comparação têm os mesmos nomes das colunas do ILS, o que permite concatenar diretamente os resultados dos dois métodos em uma análise. São exemplos `seed`, `best_objective_value`, `delta_abs_vs_baseline`, `delta_pct_vs_baseline`, `iterations`, `evaluations`, `runtime_seconds` e `stopping_reason`. Além dessas, a saída do GRASP traz colunas próprias do método, como `alpha_used` e `rcl_size_mean` por iteração, e `construction_sum_iqc` e `after_ls_sum_iqc`, que separam o valor da solução construída do valor após a busca local. Os registros em tela seguem o mesmo padrão do ILS, com linhas de início, de progresso por iteração e de encerramento por semente, controladas por variáveis de ambiente.

2.15 Parâmetros e configuração

A Tabela 4 lista os principais parâmetros da implementação, seus valores padrão e a variável de ambiente que permite alterá-los sem mexer no código. Os valores padrão foram escolhidos para coincidir com os do ILS onde os dois métodos compartilham o mesmo significado, de modo a manter a comparação sob as mesmas condições.

Tabela 4: Parâmetros da implementação do GRASP.

Parâmetro	Padrão	Variável de ambiente
Máximo de avaliações	30 000	GRASP_MAX_EVALUATIONS
Máximo de iterações	500	GRASP_MAX_ITERATIONS
Máximo sem melhora	80	GRASP_MAX_NO_IMPROVE
Amostra de vizinhos na busca local	64	GRASP_LS_NEIGHBOR_SAMPLE
Passos máximos de busca local	25	GRASP_LS_MAX_STEPS
Estratégia de α	random	GRASP_ALPHA_MODE
α fixo (quando aplicável)	0,2	GRASP_ALPHA_FIXED
Processos paralelos	automático	GRASP_SEED_PARALLEL_WORKERS

2.16 Discussão dos valores dos parâmetros

Os valores da Tabela 4 têm justificativas ligadas à teoria da Seção 1, ao custo computacional do problema e ao desenho da comparação com o ILS. Esta subseção percorre essas justificativas.

Os três limites de parada, 30 000 avaliações, 500 iterações e 80 iterações sem melhora, repetem os do ILS de propósito. A unidade de esforço é a avaliação da função objetivo porque ela é o custo dominante da execução: cada chamada recalcula os pesos CRITIC e o IQC sobre a matriz inteira, e nos registros de perfil das execuções do ILS, que usam a mesma função, essas chamadas respondem por cerca de 94% do tempo total. Um teto em avaliações também não depende da máquina nem do paralelismo. Com o mesmo teto nos dois métodos, a comparação fica definida sob esforço idêntico. O significado de iteração muda de um método para o outro — no GRASP, uma iteração é uma construção completa seguida de busca local — mas isso não afeta a contagem de esforço, porque a construção não consome avaliações: cada iteração custa uma avaliação da solução construída mais as avaliações da busca local. Sob esse custo, o teto de 30 000 permite muitas construções por semente, o que preserva o caráter multi-start do método; os limites de iterações e de estagnação funcionam como salvaguardas com a mesma semântica do ILS.

Os parâmetros da busca local, 64 vizinhos por passo e 25 passos no máximo, são idênticos aos do ILS, e a igualdade é deliberada: como o operador é o mesmo, qualquer diferença de desempenho entre os métodos fica atribuível às partes que de fato os distinguem, que são a construção com multi-start no GRASP e a perturbação no ILS. Os valores em si têm a mesma justificativa de custo. A vizinhança completa do movimento de realocação tem tamanho da ordem do produto entre as posições não nulas da solução (até 100) e as origens elegíveis (entre 167 e 194 nas instâncias do projeto), o que dá dezenas de milhares de vizinhos; uma varredura completa gastaria boa parte do orçamento em um único passo. A amostragem com primeira melhora limita o custo de uma chamada a $25 \times 64 = 1\,600$ avaliações, cerca de 5% do orçamento.

A estratégia de α sorteado uniformemente em $[0, 1]$ a cada iteração segue diretamente a recomendação da teoria: um único valor fixo de α com frequência atrapalha a busca por soluções de alta qualidade que outro valor encontraria, e o sorteio uniforme é a alternativa simples que introduz diversidade sem exigir ajuste. O GRASP Reativo, que ajusta α pelo histórico de qualidade das soluções, foi considerado e deixado de fora por acrescentar parâmetros e código sem necessidade para o objetivo da comparação; extensões desse tipo são objeto do método híbrido do projeto. O valor 0,2 do modo fixo foi mantido como opção de configuração porque é o valor citado na teoria como aquele que costuma combinar média de solução baixa com variância relativamente alta; ele só é usado quando o modo fixo é selecionado

por variável de ambiente.

Os dois pisos numéricos têm papéis distintos. O piso da faixa, 10^{-9} , entra no denominador do fator de coluna da Equação (7) e só age quando uma coluna é constante, ou seja, tem faixa nula; ele evita a divisão por zero sem alterar o escore das colunas normais, cujas faixas são ordens de grandeza maiores. A tolerância do objetivo, 10^{-12} , evita que ruído de ponto flutuante seja tratado como melhora na busca local e na atualização da melhor solução; como o IQC é arredondado a quatro casas decimais, qualquer melhora real da soma é da ordem de 10^{-4} ou maior.

O orçamento de 100 POIs não é um parâmetro do algoritmo, e sim um dado do problema. A decisão de projeto foi fazer dele o número de passos da construção, de modo que toda solução construída tenha exatamente 100 POIs e o espaço de busca coincida com o do ILS. O número de processos paralelos, por fim, é um parâmetro operacional: ele reparte as sementes entre processos, mas não altera o resultado de nenhuma semente, porque cada execução usa seu próprio gerador determinístico inicializado pela semente. As 100 sementes de `seeds.txt` são as mesmas usadas pelo ILS, o que habilita comparações pareadas semente a semente entre os métodos.

2.17 Limitações e decisões de projeto

A principal aproximação da implementação está na função gulosa da construção. O escore da Equação (7) não é o ganho exato, e sim uma estimativa que mantém os pesos CRITIC fixos e corrige a saturação apenas de forma aproximada, pela atualização das faixas por coluna. Essa foi uma escolha deliberada, justificada pela não separabilidade da função objetivo e pelo custo proibitivo da construção gulosa exata, e é coerente com a prática, descrita na teoria, de usar estimativas rápidas de valor de movimento para montar listas de candidatos. O valor real de toda solução é sempre calculado pela função objetivo verdadeira; a estimativa influencia apenas em quais pares recebem POIs durante a construção, não na medida de qualidade usada para aceitar ou comparar soluções. O exemplo da Seção 2.13 deixa isso explícito: a solução construída chegou a piorar o objetivo, e foi a busca local, guiada pela função verdadeira, que recuperou e ultrapassou a linha de base.

Uma consequência do escore separável é que hexágonos de alcance alto tendem a ser preferidos de forma persistente, já que o fator de linha alcance(h) não muda ao longo da construção. A diversidade vem de três fontes: o sorteio de α e da posição na RCL, a atualização das faixas por coluna, que redistribui as inserções entre dimensões, e o caráter multi-start com muitas sementes. A busca local, por fim, refina a solução dentro de cada dimensão. Outras estratégias descritas na Seção 1, como o GRASP Reativo e o path-relinking, ficaram fora desta versão por serem extensões que não pertencem ao template clássico; elas são objeto de um método distinto no projeto.

3 Análise dos resultados

Esta seção analisa os resultados produzidos pela implementação do GRASP. Todos os números vêm dos arquivos gravados em `results/grasp/`, e todas as figuras foram geradas a partir desses arquivos.

3.1 Desenho experimental

Os experimentos cobrem cinco localidades (`av_paulista`, `itajuba_centro`, `metro_ana_rosa`, `milao_italia`, `shinjuku_toquio`) e três perfis de caminhada (`athlete`, `average_adult`, `elderly`), totalizando quinze instâncias. Cada instância foi resolvida com as mesmas 100 sementes de `seeds.txt` usadas pelo ILS, e cada semente é uma execução completa e independente, o que dá 1 500 execuções. Todas usaram a configuração padrão: teto de 30 000 avaliações, máximo de 500 iterações, máximo de 80 iterações sem melhora, α sorteado uniformemente em $[0, 1]$ a cada iteração, e a busca local idêntica à do ILS (64 vizinhos por passo, 25 passos).

3.2 Metodologia estatística

Para cada instância, as 100 sementes são tratadas como uma amostra de 100 observações independentes do valor final. Reporta-se média, desvio padrão amostral, mediana, intervalo interquartilico, coeficiente de variação e intervalo de confiança de 95% para a média por $\bar{x} \pm t_{0,975;99} s/\sqrt{n}$. A melhoria sobre a linha de base é medida em porcentagem. A normalidade da distribuição dos valores finais entre sementes foi verificada pelo teste de Shapiro–Wilk: a hipótese de normalidade foi rejeitada ao nível de 5% em sete das quinze instâncias, sempre com assimetria positiva; por isso as medianas são reportadas junto com as médias, e os testes usados nas demais análises são não paramétricos.

Três análises específicas do método usam testes próprios. Para verificar se o parâmetro α influencia o resultado, a distribuição do α da melhor iteração de cada execução é comparada com a distribuição uniforme por um teste de Kolmogorov–Smirnov: se α fosse irrelevante, o α da melhor iteração seria uniforme. Para medir a associação entre α e a qualidade das soluções, calcula-se a correlação de postos de Spearman entre o α da iteração e o valor após a busca local, separadamente dentro de cada execução (as iterações de uma mesma execução não são independentes entre execuções distintas, então a execução é a unidade de análise); reporta-se a média das 100 correlações por instância, com intervalo de confiança, e um teste de Wilcoxon da hipótese de correlação nula. Para a forma da distribuição do IQC por hexágono, a assimetria e o excesso de curtose amostrais (g_1 e g_2 , com correção de viés) são acompanhados dos testes de D’Agostino para assimetria e curtose.

3.3 Resultados descritivos

A Tabela 5 apresenta as estatísticas por instância e a Figura 3 mostra a distribuição da melhoria porcentual sobre a base, por semente. Em todas as quinze instâncias, todas as 100 sementes terminaram acima da linha de base. A dispersão entre sementes é baixa: o coeficiente de variação fica entre 0,19% e 2,11%. O padrão de dependência da instância repete o já observado no projeto: a melhoria é grande onde a linha de base é baixa (Itajubá, entre 43 e 51%) e pequena onde a base já é alta (Milão, entre 3,3 e 6,1%).

Tabela 5: Estatísticas descritivas do GRASP por instância (100 sementes). Valores em soma de IQC; $\Delta IQC\%$ é a melhoria média sobre a base; CV% é o coeficiente de variação; tempo em segundos.

Localidade	Perfil	Base	Média	Desvio	Mediana	$\Delta IQC\%$	CV%	Tempo m
av_paulista	athlete	54,00	60,21	0,374	60,17	11,50	0,62	26,3
av_paulista	average_adult	47,06	53,43	0,461	53,41	13,55	0,86	18,3
av_paulista	elderly	38,68	44,87	0,419	44,79	16,01	0,93	13,8
itajuba_centro	athlete	30,23	43,27	0,710	43,26	43,12	1,64	19,9
itajuba_centro	average_adult	25,41	38,40	0,811	38,43	51,12	2,11	18,6
itajuba_centro	elderly	22,75	32,99	0,691	32,93	45,00	2,10	13,7
metro_ana_rosa	athlete	42,54	53,15	0,604	53,11	24,93	1,14	32,3
metro_ana_rosa	average_adult	38,53	47,77	0,573	47,77	23,97	1,20	20,5
metro_ana_rosa	elderly	32,45	37,06	0,378	37,07	14,23	1,02	14,4
milao_italia	athlete	62,95	64,99	0,126	65,00	3,25	0,19	31,7
milao_italia	average_adult	57,45	59,90	0,267	59,84	4,27	0,45	25,4
milao_italia	elderly	49,55	52,60	0,138	52,60	6,15	0,26	16,3
shinjuku_toquio	athlete	57,10	60,25	0,189	60,22	5,52	0,31	28,5
shinjuku_toquio	average_adult	51,27	55,10	0,305	55,06	7,48	0,55	23,6
shinjuku_toquio	elderly	42,23	46,74	0,357	46,70	10,68	0,76	18,3

3.4 Comportamento do parâmetro α

Esta é a análise mais informativa sobre o método, porque o α é o único parâmetro genuíno do GRASP clássico. Duas perguntas orientam a análise: o α influencia o resultado? E, se influencia, existe um valor bom universal?

Para a primeira pergunta, considere o α da iteração que produziu a melhor solução de cada execução. Se o α fosse irrelevante, esse valor seria uniforme em $[0, 1]$, já que os α são sorteados de maneira uniforme. O

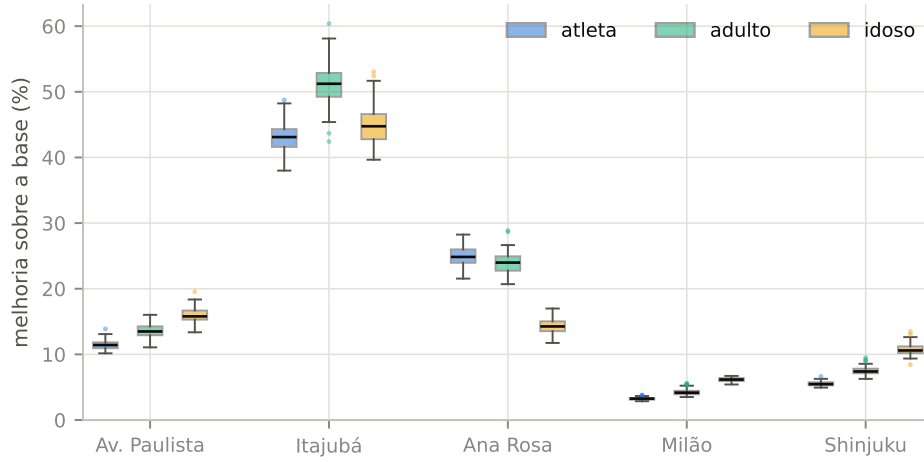


Figura 3: Distribuição, entre as 100 sementes, da melhoria porcentual do GRASP sobre a linha de base, por instância. Caixas indicam quartis, a linha central é a mediana, e os pontos isolados são valores fora de 1,5 vez o intervalo interquartílico.

teste de Kolmogorov–Smirnov rejeita a uniformidade com folga ($D = 0,152$, $p \approx 1,2 \times 10^{-30}$, $n = 1\,500$): o painel esquerdo da Figura 4 mostra a concentração em torno do centro do intervalo e o esvaziamento dos extremos. O α importa.

Para a segunda pergunta, o painel direito da Figura 4 mostra a média, por instância, do α da melhor iteração. O intervalo é amplo: vai de 0,15 em Milão (adulto e idoso) a 0,87 em Itajubá (atleta). Ou seja, não existe um valor bom universal: em algumas instâncias as melhores soluções vêm de construções quase gulosas, em outras vêm de construções quase aleatórias. Esse resultado confirma nos dados a recomendação teórica da Seção 1, de que um único α fixo com frequência atrapalha, e sustenta a escolha do sorteio uniforme feita nesta implementação: o sorteio permite que cada instância encontre a sua faixa boa de α sem nenhum ajuste prévio.

A Tabela 6 quantifica a associação entre α e qualidade dentro das execuções, pela correlação de Spearman média entre o α da iteração e o valor da solução após a busca local. Todas as correlações médias são diferentes de zero (Wilcoxon, $p < 10^{-15}$ em todas as instâncias), mas o sinal muda: a correlação é positiva e forte em Itajubá (até +0,95), moderada na maior parte das instâncias, e negativa em Milão (até −0,48 no perfil idoso). A Figura 5 mostra os dois padrões extremos: em Itajubá, quanto mais aleatória a construção, melhor a solução; em Milão, a curva sobe rápido até α por volta de 0,15 e decai a partir daí, favorecendo construções quase gulosas.

O padrão tem uma explicação ligada à estrutura do problema. O score surrogate favorece persistentemente as origens de maior alcance; com α pe-

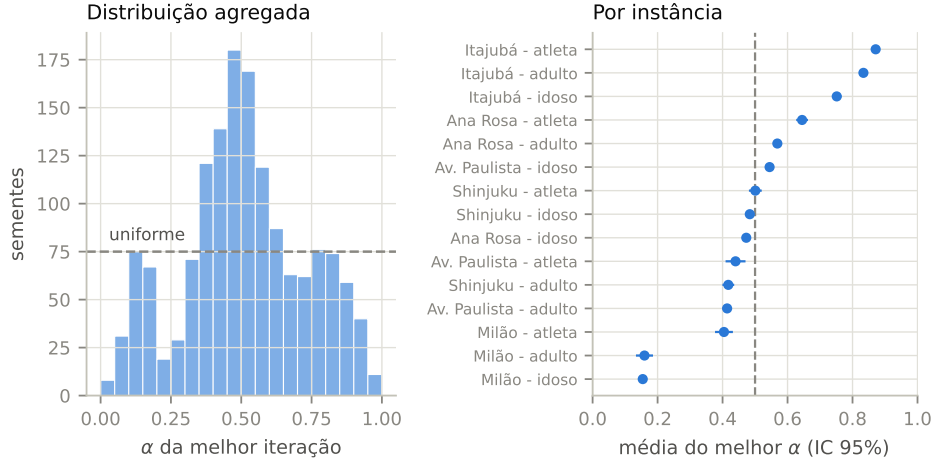


Figura 4: À esquerda, distribuição agregada do α da melhor iteração nas 1 500 execuções, com a referência uniforme tracejada; a uniformidade é rejeitada por Kolmogorov–Smirnov ($p \approx 10^{-30}$). À direita, média por instância com intervalo de confiança de 95%: o melhor α varia de 0,15 a 0,87 conforme a instância.

queno, as inserções se concentram nessas origens. Em instâncias com poucas origens de alcance alto e base baixa, como Itajubá, essa concentração eleva demais o máximo das colunas e, pela normalização min–max do IQC, rebaixa os valores normalizados dos demais hexágonos — o mesmo mecanismo visto no exemplo da Seção 2.13. Nessas instâncias, construções mais aleatórias espalham os POIs e produzem valores reais melhores. Em instâncias grandes e de base alta, como Milão, o efeito de concentração é diluído e a informação do escore guloso volta a valer.

Tabela 6: Análise do α por instância: média e mediana do α da melhor iteração, e correlação de Spearman média (entre iterações de uma execução, média sobre as 100 execuções, com IC de 95%) entre o α e o valor após a busca local. Todas as correlações diferem de zero (Wilcoxon, $p < 10^{-15}$).

Localidade	Perfil	$\bar{\alpha}$ melhor	Mediana	$\bar{\rho}_s(\alpha, F_{LS})$	IC 95%
av_paulista	athlete	0,440	0,354	+0,290	[+0,267; +0,313]
av_paulista	average_adult	0,414	0,399	+0,423	[+0,403; +0,443]
av_paulista	elderly	0,545	0,524	+0,606	[+0,590; +0,623]
itajuba_centro	athlete	0,871	0,880	+0,950	[+0,946; +0,953]
itajuba_centro	average_adult	0,834	0,827	+0,915	[+0,910; +0,919]

Localidade	Perfil	$\bar{\alpha}$ melhor	Mediana	$\bar{\rho}_s(\alpha, F_{LS})$	IC 95%
itajuba_centro	elderly	0,751	0,753	+0,805	[+0,797; +0,814]
metro_ana_rosa	athlete	0,645	0,659	+0,593	[+0,580; +0,607]
metro_ana_rosa	average_adult	0,569	0,565	+0,530	[+0,513; +0,547]
metro_ana_rosa	elderly	0,473	0,461	+0,634	[+0,620; +0,648]
milao_italia	athlete	0,404	0,431	+0,078	[+0,059; +0,097]
milao_italia	average_adult	0,160	0,121	-0,277	[-0,297; -0,258]
milao_italia	elderly	0,154	0,158	-0,485	[-0,504; -0,466]
shinjuku_toquio	athlete	0,501	0,498	+0,416	[+0,397; +0,436]
shinjuku_toquio	average_adult	0,418	0,405	+0,217	[+0,198; +0,235]
shinjuku_toquio	elderly	0,484	0,483	+0,324	[+0,302; +0,347]

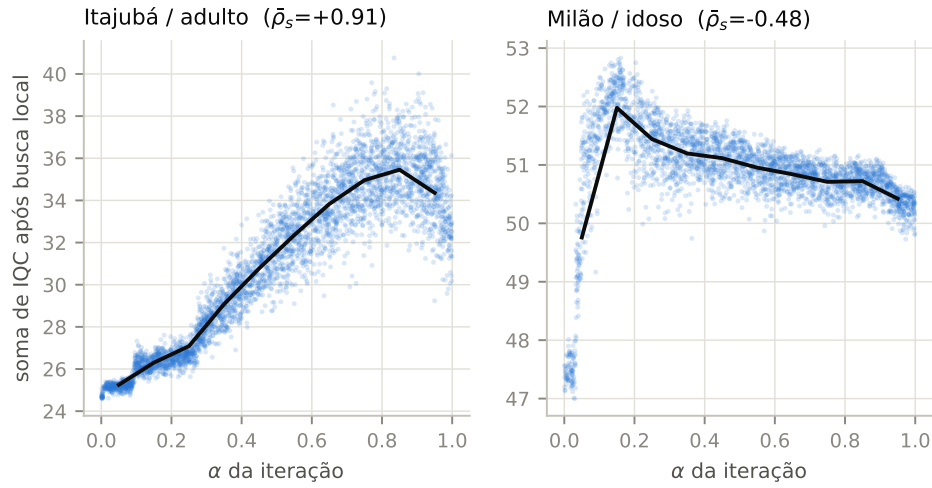


Figura 5: Valor após a busca local em função do α da iteração (pontos: iterações de 25 sementes; linha: média por faixa de α), nas duas instâncias de comportamento extremo. Em Itajubá a qualidade cresce com α ; em Milão há um pico perto de $\alpha = 0,15$ seguido de queda.

3.5 Construção versus busca local

A Tabela 7 separa a contribuição das duas fases. A coluna da melhor construção é o maior valor de solução construída (antes da busca local) da execução, em média sobre as sementes; a diferença até o melhor valor final mede o que a busca local acrescenta além da melhor construção pura. A busca local responde por 21 a 38% do ganho total sobre a base, com o restante vindo da

construção — ou seja, a construção gulosa randomizada com escore surrogate produz sozinha a maior parte do ganho, e a busca local refina de forma consistente. A Figura 6 mostra o efeito no nível da iteração: todos os pontos ficam sobre ou acima da reta identidade, porque a busca local por primeira melhora nunca piora a solução recebida.

Tabela 7: Decomposição do ganho entre construção e busca local (médias sobre 100 sementes, em soma de IQC). A última coluna é a fração do ganho total sobre a base devida à busca local.

Localidade	Perfil	Melhor construção	Melhor final	Acréscimo da BL	BL no ga
av_paulista	athlete	58,18	60,21	2,03	32
av_paulista	average_adult	51,53	53,43	1,90	29
av_paulista	elderly	43,39	44,87	1,48	23
itajuba_centro	athlete	39,41	43,27	3,86	29
itajuba_centro	average_adult	35,01	38,40	3,39	26
itajuba_centro	elderly	30,52	32,99	2,47	24
metro_ana_rosa	athlete	49,94	53,15	3,21	30
metro_ana_rosa	average_adult	45,03	47,77	2,74	29
metro_ana_rosa	elderly	35,77	37,06	1,30	28
milao_italia	athlete	64,31	64,99	0,69	33
milao_italia	average_adult	59,11	59,90	0,79	32
milao_italia	elderly	51,97	52,60	0,63	20
shinjuku_toquio	athlete	59,04	60,25	1,21	38
shinjuku_toquio	average_adult	53,81	55,10	1,29	33
shinjuku_toquio	elderly	45,47	46,74	1,27	28

O tamanho médio da RCL, registrado por iteração, ficou entre 197 e 414 elementos conforme a instância, sobre uma grade de aproximadamente 1 200 a 1 360 candidatos. A RCL é relativamente grande porque o escore surrogate é separável (produto de um fator de linha por um fator de coluna), o que gera muitos escores próximos entre si.

3.6 Critério de parada e custo computacional

Todas as 1 500 execuções pararam por estagnação, ao atingir 80 iterações sem melhora da melhor solução; nenhuma esgotou o orçamento de avaliações nem o limite de iterações. As execuções fizeram em média entre 137 e 155 iterações e gastaram entre 7 800 e 11 700 avaliações, ou seja, entre 26 e 39% do teto de 30 000. Cada iteração custou em média de 55 a 78 avaliações (uma

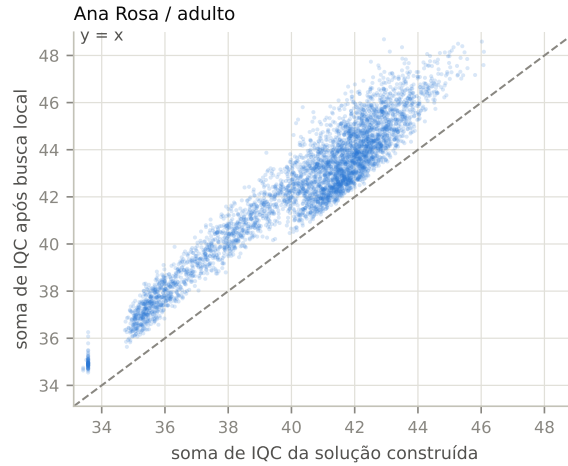


Figura 6: Valor da solução construída contra o valor após a busca local, por iteração (30 sementes de Ana Rosa / adulto). Nenhum ponto fica abaixo da reta identidade: a busca local por primeira melhora nunca piora a solução.

da solução construída, o restante da busca local). O tempo por semente ficou entre 14 e 32 segundos.

Esse comportamento é o esperado para o template clássico. Como o GRASP é uma amostragem repetitiva sem memória entre iterações, a probabilidade de uma nova iteração superar a melhor solução acumulada decai conforme a execução avança; a partir de certo ponto, as construções continuam produzindo soluções boas, mas raramente recordes. O critério de estagnação detecta esse regime e encerra a execução, evitando gastar o restante do orçamento em iterações com retorno esperado baixo.

3.7 Convergência

A Figura 7 e a Tabela 8 mostram a convergência nas duas instâncias representativas usadas ao longo do capítulo. O perfil é o típico de multi-start: ganho muito rápido nas primeiras iterações, quando quase toda construção melhora o recorde, seguido de saturação. Por volta de 1 000 avaliações já se alcança cerca de 85% da melhoria total; em 5 000, cerca de 96 a 97%; a partir de 10 000 o ganho residual é inferior a 1%.

3.8 Distribuição do IQC por hexágono: assimetria e curtose

As análises anteriores olham para a soma do IQC. Esta subseção olha para a forma da distribuição do IQC entre os hexágonos, antes e depois da intervenção, aplicando a melhor alocação global de cada instância e recalculando o IQC de todos os hexágonos. A assimetria (g_1) e o excesso de curtose (g_2)

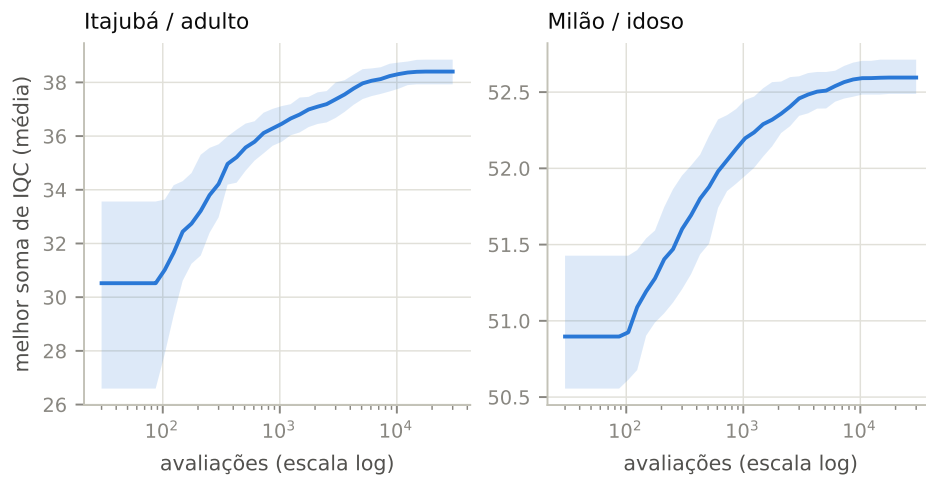


Figura 7: Convergência do GRASP: média, sobre as 100 sementes, do melhor valor em função do número de avaliações (linha) e faixa interquartílica (área sombreada), em escala logarítmica de avaliações.

Tabela 8: Convergência: média do melhor valor por número de avaliações, e fração da melhoria total (entre parênteses).

Instância	100	1.000	5.000	10.000	30.000
Itajubá / adulto	30,82 (42%)	36,42 (85%)	37,94 (96%)	38,31 (99%)	38,40 (100%)
Milão / idoso	50,91 (45%)	52,18 (86%)	52,51 (97%)	52,59 (100%)	52,60 (100%)

amostrais são acompanhados dos testes de D’Agostino; a Tabela 9 traz os valores, e as Figuras 8 e 9 os ilustram.

Antes da intervenção, doze das quinze instâncias têm assimetria positiva significativa ao nível de 5% (valores de g_1 entre +0,45 e +1,37): a massa da distribuição concentra-se nos hexágonos de IQC baixo, com uma cauda de poucos hexágonos bons. Depois da intervenção, a assimetria cai em quatorze das quinze instâncias, e permanece significativa em apenas sete; o excesso de curtose também cai na maioria delas, com os pontos migrando para perto da região $(g_1, g_2) \approx (0, \text{negativo})$ na Figura 9, que corresponde a distribuições mais simétricas e mais achatadas que a normal. Milão é a exceção que confirma o mecanismo: a distribuição já era aproximadamente simétrica antes (g_1 entre $-0,09$ e $+0,13$, não significativos) e muda pouco.

A leitura substantiva é que a otimização não apenas aumenta a soma do IQC: ela desconcentra a distribuição. Como o objetivo é a soma e a normalização é min-max, elevar hexágonos da cauda baixa rende mais do que reforçar hexágonos já bons, e o resultado agregado é uma região com caminhabilidade mais homogênea, não apenas maior.

Tabela 9: Assimetria (g_1) e excesso de curtose (g_2) da distribuição do IQC por hexágono, antes e depois da melhor alocação do GRASP. O símbolo † marca valores significativamente diferentes de zero pelo teste de D’Agostino ao nível de 5%.

Localidade	Perfil	g_1 antes	g_1 depois	g_2 antes	g_2 depois
av_paulista	athlete	+0,45 †	+0,06	-0,37	-0,89 †
av_paulista	average_adult	+0,52 †	+0,10	-0,24	-1,06 †
av_paulista	elderly	+0,58 †	+0,28	-0,07	-0,60 †
itajuba_centro	athlete	+0,82 †	+0,44 †	-0,34	-0,66 †
itajuba_centro	average_adult	+1,08 †	+0,50 †	+0,34	-0,35
itajuba_centro	elderly	+1,37 †	+0,66 †	+1,48 †	-0,07
metro_ana_rosa	athlete	+0,76 †	+0,25	+0,33	-0,20
metro_ana_rosa	average_adult	+0,92 †	+0,30	+0,85	-0,34
metro_ana_rosa	elderly	+1,01 †	+0,79 †	+1,03 †	+0,14
milao_italia	athlete	-0,09	-0,15	-0,93 †	-0,92 †
milao_italia	average_adult	+0,01	-0,13	-0,84 †	-0,99 †
milao_italia	elderly	+0,13	-0,00	-0,62 †	-0,82 †
shinjuku_toquio	athlete	+0,63 †	+0,44 †	-0,08	-0,21
shinjuku_toquio	average_adult	+0,77 †	+0,49 †	+0,33	-0,13
shinjuku_toquio	elderly	+0,88 †	+0,56 †	+0,51	-0,25

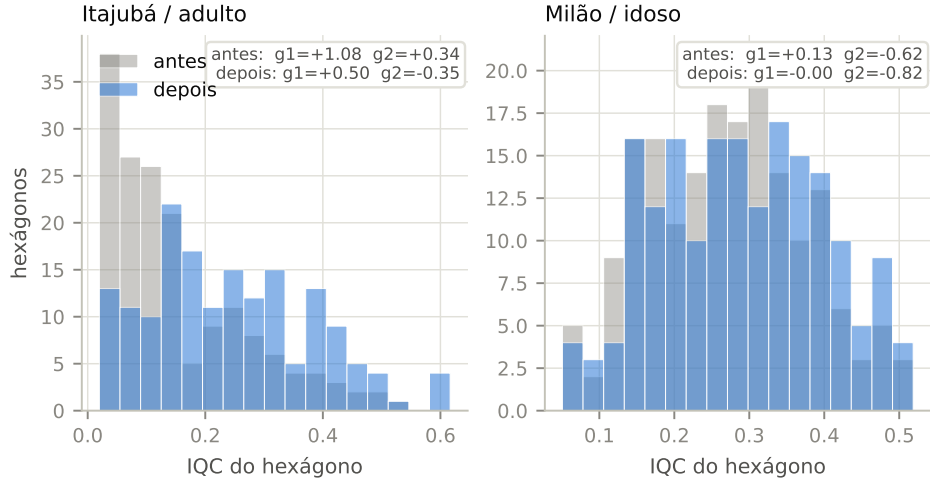


Figura 8: Distribuição do IQC por hexágono antes (cinza) e depois (azul) da melhor alocação do GRASP, nas duas instâncias representativas, com g_1 e g_2 anotados. Em Itajubá a cauda baixa esvazia e a distribuição se espalha para a direita; em Milão, já simétrica, a mudança é pequena.

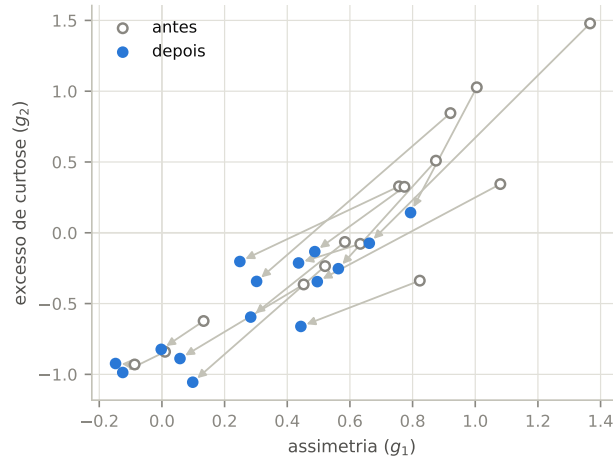


Figura 9: Migração das quinze instâncias no plano assimetria $\times$ excesso de curtose, da distribuição de base (círculos vazios) para a distribuição após a melhor alocação do GRASP (círculos cheios). O movimento predominante é para a esquerda e para baixo: distribuições mais simétricas e menos concentradas.

3.9 Síntese

Os dados sustentam cinco conclusões. Primeiro, o GRASP melhora a linha de base em todas as instâncias e todas as sementes, com dispersão baixa entre sementes (coeficiente de variação máximo de 2,1%), o que indica robustez. Segundo, o parâmetro α importa e seu valor bom depende da instância — o α da melhor iteração vai de 0,15 a 0,87 conforme o caso, e a correlação entre α e qualidade muda até de sinal —, o que confirma nos dados a escolha do sorteio uniforme de α e a advertência da teoria contra valores fixos. Terceiro, a construção responde pela maior parte do ganho (de 62 a 79%) e a busca local refina o restante, sem nunca piorar. Quarto, o método converge cedo e para por estagnação em todas as execuções, gastando cerca de um terço do orçamento de avaliações; isso é coerente com o caráter de amostragem repetitiva sem memória do template clássico. Quinto, além de aumentar a soma do IQC, a intervenção torna a distribuição do IQC entre hexágonos mais simétrica e menos concentrada — a assimetria positiva significativa presente em doze instâncias antes da intervenção cai em quatorze delas depois —, o que na prática significa uma melhora mais bem distribuída pela região, e não restrita a poucos hexágonos.

Referências

- [1] M. G. C. Resende e C. C. Ribeiro. GRASP. AT&T Labs Research Technical Report, 2008.